

LAB EXERCISE 7

CODE INJECTION ON APPLICATION PROCESS USING PTRACE

AIM

To study and perform code injection on a running application process using the ptrace system call in Kali Linux for understanding process control and debugging mechanisms.

SOFTWARE / TOOLS REQUIRED

Kali Linux
GCC compiler
Root privileges (sudo)
ptrace (Linux system call)

PROCEDURE

Step 1: Create a Target Program

This program runs continuously so it can be attached to.

```
// target.c
#include <stdio.h>
#include <unistd.h>

int main() {
    while (1) {
        printf("Target process running...\n");
        sleep(2);
    }
    return 0;
}
```

Compile and run:

```
gcc target.c -o target
./target
```

Step 2: Find Process ID

Open a new terminal and find the PID.

```
ps aux | grep target
```

Example:

```
root 2456 0.0 ./target
```

PID = 2456

Step 3: Create Ptrace Injection Program

This program attaches to the target and modifies its instruction pointer (RIP).

```
// inject.c
#include <sys/ptrace.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <pid>\n", argv[0]);
        return 1;
    }

    pid_t pid = atoi(argv[1]);
    struct user_regs_struct regs;

    ptrace(PTRACE_ATTACH, pid, NULL, NULL);
    wait(NULL);

    ptrace(PTRACE_GETREGS, pid, NULL, &regs);
    printf("Original RIP: %llx\n", regs.rip);

    regs.rip += 2; // simple instruction pointer modification
    ptrace(PTRACE_SETREGS, pid, NULL, &regs);

    ptrace(PTRACE_DETACH, pid, NULL, NULL);
    return 0;
}
```

Compile:

```
gcc inject.c -o inject
```

Step 4: Execute Injection

Run with root privileges:

```
sudo ./inject 2456
```

The injector attaches, modifies execution flow, and detaches.

OUTPUT

Injector successfully attaches to the target process.

Instruction pointer (RIP) is modified.

Target process execution behavior changes momentarily.

```
(kali㉿kali)-[~]  
$ nano target.c  
  
(kali㉿kali)-[~]  
$ gcc target.c -o target  
  
(kali㉿kali)-[~]  
$ ./target  
Target process running ...  
Target process running ...  
Target process running ...  
Target process running ...  
Target process running ...
```

[illegible]

RESULT

Thus, code injection using ptrace was successfully performed on a running application process in Kali Linux.

This experiment demonstrates process tracing, register manipulation, and runtime control using the ptrace system call.